# EduOS: A Linux-Based Academic Computing Platform

Project Proposal for UCS503P

Submitted to: **Dr. Raghav B. Venkataramaiyer**
Thapar Institute of Engineering and Technology

**Dhawal**
CSED
Roll No: XXXXXXX
Email: XXXXXXX

August 18, 2026

## 1 Introduction

Modern academic laboratories rely heavily on software. Programming courses require different compilers and development environments, database courses require database systems and clients, networking courses require specialized tools, and operating-system courses often require access to low-level system utilities.

Although the underlying hardware in a laboratory may be identical, the software environment can vary significantly between machines. Installing packages manually, correcting broken configurations, maintaining dependencies, and restoring machines after misuse can consume a considerable amount of administrative time.

EduOS proposes an alternative approach: instead of treating laboratory computers as independent general-purpose machines, they are treated as managed instances of a common academic computing environment.

EduOS will be developed as a Gentoo-based Linux distribution with an accompanying management and assessment ecosystem. The project will combine a customized operating system, declarative laboratory profiles, centralized machine management, an academic software hub, and automated laboratory evaluation.

## 2 Vision of EduOS

The central idea behind EduOS is simple:

*The institution defines the computing environment; EduOS makes every machine conform to it.*

Rather than manually configuring each laboratory computer, an administrator should be able to describe the requirements of a laboratory through a profile.

For example, a C++ programming laboratory may require:

- GCC and Clang.
- CMake.
- GDB.
- Git.
- Required libraries.
- Course-specific tools.

A networking laboratory may instead require packet-analysis tools, network utilities, simulation software, and specific system configurations.

EduOS will use these profiles to determine what software and configuration a machine should have. The system will then continuously work toward that desired state.

This transforms laboratory administration from a collection of manual installation procedures into a reproducible configuration-management problem.

# 3 Problem Being Addressed

A university laboratory generally has three separate concerns:

1. Providing students with the correct computing environment.
2. Maintaining that environment across many machines.
3. Evaluating the work produced inside that environment.

These concerns are normally handled by different tools and manual processes.

Students may encounter problems such as:

- A required package is missing.
- A compiler or interpreter version differs from the expected version.
- Dependencies have been installed incorrectly.
- A laboratory machine has been modified by another user.
- The student's local environment behaves differently from the instructor's environment.

Instructors face a different problem. Even after providing the environment, laboratory submissions may still need to be manually compiled, executed, tested, and checked.

EduOS attempts to address both sides of this problem by combining environment management and laboratory evaluation into one platform.

# 4 Proposed System

EduOS will consist of several interconnected components rather than being limited to a customized Linux ISO.

## 4.1 EduOS Operating System

The operating-system layer will be based on Gentoo Linux.

Gentoo is particularly suitable for this project because its package and profile system allows the project to define a highly customized environment instead of starting with a fixed collection of packages.

The EduOS image will contain the common software required by students and administrators, while course-specific software can be introduced through laboratory profiles.

The operating system will also be designed with the academic use case in mind. Unnecessary services can be removed, while development tools, documentation, utilities, and educational resources can be provided as part of the platform.

## 4.2 Laboratory Profiles

A laboratory profile will describe the desired state of a particular academic environment.

A profile may contain:

- Operating-system packages.
- Compiler and interpreter requirements.
- Development libraries.
- System services.
- Configuration files.
- Environment variables.
- Course-specific applications.
- Policies or restrictions.

Profiles will be represented using structured configuration files and stored under version control.

This provides an important property for the project: an entire laboratory environment can be recreated from a known configuration instead of relying on undocumented manual setup steps.

### 4.3 EduOS Management Server

The management server will act as the central control point for the system.

Administrators will be able to:

- Register machines.
- Assign machines to laboratories.
- Create and modify laboratory profiles.
- Monitor machine status.
- View synchronization information.
- Manage available academic software.

The server will expose an API through which managed machines communicate with the central system.

### 4.4 EduOS Agent

Each managed machine will run a lightweight EduOS agent.

The agent will periodically communicate with the management server and determine whether the local system matches the assigned laboratory profile.

A simplified model is:

**Desired State → Agent → Local System**

If a required package is missing, the agent can initiate the appropriate package-management operation. If a configuration has changed, the system can restore the expected configuration.

This approach also provides a mechanism for detecting configuration drift.

## 5 EduLab: Academic Assessment Layer

A major part of EduOS is the integration of laboratory assessment.

EduLab will provide a common interface through which students can receive laboratory tasks, submit their work, and view evaluation results.

Instead of creating a separate evaluation system for every subject, EduLab will use a common submission format and connect submissions to specialized evaluators.

The proposed architecture is:

**Student → EduLab → Evaluator → Result**

## 5.1 Programming Evaluation

For programming laboratories, a submission may contain source code and supporting files.

The evaluator will:

1. Create an isolated execution environment.
2. Compile the student's program.
3. Execute predefined test cases.
4. Apply resource limits.
5. Collect program output and execution information.
6. Compare the result with the expected output.
7. Generate a score and evaluation report.

This allows a C/C++ laboratory, for example, to be evaluated automatically without requiring the instructor to manually compile every submission.

## 5.2 Multiple Laboratory Domains

The evaluator system will be modular.

The first implementation can focus on programming laboratories, while the architecture will allow future evaluators for:

- C/C++ programming.
- SQL and database exercises.
- Linux administration.
- Computer networking.
- Shell scripting.
- Other command-line based laboratory exercises.

This makes EduLab a general laboratory framework rather than a single-purpose programming judge.

# 6 System Architecture

The proposed EduOS architecture can be divided into four major layers.

| Layer | Purpose |
|---|---|
| EduOS Client | Operating system, local tools, machine agent and student environment |
| Management Layer | Authentication, machine registration, profiles and desired-state management |
| EduLab Layer | Laboratory tasks, submissions, evaluation and result generation |
| Infrastructure Layer | Database, containers, virtualization and deployment infrastructure |

The expected communication model is:

**EduOS Machines ↔ Management API ↔ Database**

**Students → EduLab → Evaluation Workers**

The separation between these components will allow the project to evolve without requiring the entire system to be redesigned whenever a new laboratory or evaluator is introduced.

# 7 Technology Stack

The project will make use of existing technologies wherever they provide reliable infrastructure.

- **Gentoo Linux:** Base operating system.
- **Portage:** Package management and dependency handling.
- **Gentoo Profiles and Overlays:** Customization of academic environments.
- **Catalyst:** Automated generation of EduOS system images.
- **Python:** Backend services and system tooling.
- **FastAPI:** Management and EduLab APIs.
- **PostgreSQL:** Persistent application data.
- **React and TypeScript:** Web interfaces.
- **Ansible:** Configuration and machine-management support.
- **Podman:** Isolation of evaluation workloads.
- **QEMU/KVM:** Virtual testing of operating-system images.
- **Git:** Version control for software and laboratory configurations.
- **YAML/Pydantic:** Structured and validated laboratory specifications.

The use of these components allows the project to concentrate on integrating them into an academic platform instead of implementing basic infrastructure from scratch.

# 8 Security Considerations

Security is particularly important because EduLab will execute student-created programs.

A student's program should not receive unrestricted access to the machine running the evaluator.

Therefore, submitted programs will be executed inside isolated containers with restrictions on:

- CPU usage.
- Memory usage.
- Execution time.
- Filesystem access.
- Network access.
- Available system capabilities.

The evaluator will communicate only the information required to produce the result.

The management system will also use authentication and role separation so that administrative operations are not exposed to ordinary student accounts.

# 9 Reproducibility and Version Control

One of the main advantages of the proposed system is that an academic environment becomes an explicitly defined artifact.

A laboratory profile can be stored alongside the source code of the project and assigned a version.

For example:

$$\texttt{CSE-LAB-CPP-v1} \rightarrow \text{Compiler} + \text{Libraries} + \text{Tools} + \text{Configuration}$$

If a laboratory configuration changes, the change can be recorded as a new version.

This provides a historical record of the environment under which a laboratory was conducted and makes it possible to recreate previous environments when required.

# 10 Development Methodology

The project will be developed incrementally.

### 10.1 Phase 1: EduOS Base

- Build the initial Gentoo-based EduOS image.
- Configure the base academic environment.
- Create the initial package/profile structure.
- Test the image using QEMU/KVM.

### 10.2 Phase 2: Machine Management

- Implement machine registration.
- Develop the EduOS agent.
- Create laboratory profile definitions.
- Implement desired-state synchronization.

### 10.3 Phase 3: EduLab

- Develop student submission functionality.
- Implement the first programming evaluator.
- Introduce containerized execution.
- Generate evaluation results.

### 10.4 Phase 4: Integration

The final phase will connect the operating-system, management, and assessment components and evaluate the complete workflow.

## 11 Evaluation Plan

The project will not be evaluated only on whether the software runs. The evaluation will measure whether EduOS actually improves the management and assessment workflow.

The following experiments will be performed.

### 11.1 Environment Deployment

A clean machine or virtual machine will be configured using the EduOS workflow.

The time and number of manual steps required will be compared against a conventional manual setup process.

### 11.2 Configuration Drift

A managed machine will be deliberately modified by installing or removing software.

The system will then be observed to determine whether the machine detects and corrects the deviation from its assigned profile.

### 11.3 Reproducibility Test

The same laboratory profile will be used to generate multiple environments.

The resulting environments will be compared to determine whether the required software and configurations remain consistent.

### 11.4 Automated Evaluation

A collection of known-correct, known-incorrect, compilation-failing, and resource-intensive submissions will be passed through EduLab.

The evaluator will be measured for:

- Correctness of grading.
- Execution time.
- Failure detection.
- Resource isolation.

## 12 Expected Outcomes

At the end of the project, the expected result is a functional proof of concept demonstrating that EduOS can serve as a common academic computing platform.

The system should demonstrate:

- A reproducible Gentoo-based academic operating system.
- Central management of laboratory configurations.
- Automatic synchronization of managed machines.
- A common academic software environment.
- A functional laboratory submission platform.
- Automated and isolated evaluation of programming assignments.
- Version-controlled laboratory environments.
- A modular architecture that can support additional laboratory domains.

The project is intentionally scoped as a proof of concept. The objective is not to immediately replace an institution's existing infrastructure, but to demonstrate that operating-system management and academic assessment can be integrated into a single platform.

# 13 Potential Extensions

If the initial system proves successful, several capabilities could be added in future iterations.

- A complete graphical software center for academic applications.
- Automatic machine health monitoring.
- Remote recovery or re-imaging of laboratory systems.
- Course-specific desktop environments.
- Instructor-created laboratory templates.
- Automatic generation of laboratory environments from course specifications.
- More advanced networking and systems laboratories using virtual machines.
- Integration with existing university authentication systems.
- Student performance analytics.
- Centralized update scheduling for laboratory machines.

# 14 Risks and Challenges

## 14.1 Complexity of Building a Custom Distribution

Maintaining a customized Gentoo environment introduces additional work compared with using a conventional distribution.

This will be addressed by keeping the initial package set limited and maintaining the configuration through version-controlled profiles and automated builds.

## 14.2 Hardware Compatibility

A Linux distribution intended for laboratory machines must support the available hardware.

Virtual machines will be used extensively during development, followed by testing on representative physical hardware.

### 14.3 Maintaining Consistency

Machines may be modified locally by users or affected by package changes.

The management agent and desired-state model will provide a mechanism for detecting and correcting such deviations.

### 14.4 Security of Automated Evaluation

Running arbitrary student programs presents a security risk.

Container isolation, resource restrictions, and controlled evaluator environments will therefore be part of the design rather than being added as an afterthought.

### 14.5 Project Scope

EduOS combines operating-system development, configuration management, backend development, web development, and automated evaluation.

To keep the project achievable, the initial implementation will focus on one complete end-to-end workflow instead of attempting to support every possible academic laboratory.

## 15 Conclusion

EduOS proposes a different way of approaching university computer laboratories. Instead of viewing a laboratory computer as an independently configured PC, EduOS treats it as a managed component of an institution-wide academic computing environment.

The project combines a Gentoo-based operating system with laboratory profiles, centralized configuration management, an academic software environment, and EduLab, a platform for automated laboratory assessment.

The most important property of the system is reproducibility. A laboratory environment should be definable, versionable, deployable, and recoverable without depending on a long sequence of undocumented manual configuration steps.

By combining operating-system customization with centralized management and automated evaluation, EduOS aims to demonstrate a practical platform that can reduce laboratory administration overhead while giving students a more consistent development environment and giving instructors a more systematic method of evaluating technical coursework.

The semester project will focus on implementing and validating this core concept through a working proof of concept rather than attempting to deliver a complete production-ready university operating system.